

Exercise 4: Control Statements and Formatting

Note: Before you start this exercise, if you already have Octave running and some variables defined, remove them with the “clear” command. Make a folder for today’s work in your Octave working directory, with a dated label in YYMMDD format. Remember to make notes in your research notebook as well!

1. Control Statements

1.1 For Loops

Sometimes we will want to create a very big array, or a one dimensional array with lots of elements. Typing out commands to define these types of variables is a bad strategy. It takes forever, and is prone to typos. To make many assignments in a consistent way, we can use a command called a “for loop”. Let’s create a script to create define a 1D array with 50 elements, where element 1 has a value of 1, and each subsequent element increases in value by two. Use this template:

```
%file created by XXX on YYMMDD
%tutorial exercise 3.2

clear

for index_array = 1:50
    vector(index_array) = 1 + index_array*2;
endfor
```

The new “for” command in line 4 tells Octave that we want to run a series of similar commands. The “index_array =” part of the line tells Octave that the value of a variable called index_array will be changing during each iteration of this series. The “1:50” part tells Octave that, for the first iteration, the value of index_array should be one. Each successive iteration of the loop will automatically increase the value of index_array by one. The computer will run the loop until index_array reaches a value of 50, then stop. The commands with index_array = 50 will run. Next we write the commands we want to iterate. We can have multiple lines of commands, but for now we just have the one specified on line 5, “vector(index_array) = 1 + index_array*2;”. This command assigns a value to the element of the variable named vector, with the element being specified by the value of the variable index_array. We indent this line to make it clear that

this command is inside our for loop (more on this later). In line 6 we give the “endfor” command, which lets Octave know which commands are included in our for loop. We can actually get away with just writing “end”, but endfor is useful because it is clear about what kind of control statement we are terminating. Any commands appearing after the end command would not be iterated. A more general diagram of the structure of a for loop is:

```
for (index variable) = (values to loop over)  
  command 1 to iterate  
  command 2 to iterate  
  etc...  
endfor
```

To get some understanding of how for loops really work, we need play around with our example a bit. You may have noticed that the example code provided above does not actually produce the requested output. For example, vector(1) does not have a value of 1. Can you modify your for loop to get the requested output? Can you write a nested set of two for loops with different index variables that define the elements of a 2D array? In what sequence do your two nested for loops iterate their respective index variables while assigning values to the 2D array?

There are other kinds of loop-based control statements, such as while loops, do-until statements, and switch-case statements. All of these are standard parts of most computer programming languages, and form the basis of just about all computer programs. While loops and do-until statements both execute repeatedly until a certain condition is fulfilled. For example, most computer games are basically while loops, which execute until a certain condition is fulfilled (when your score goes above a certain number, the timer runs out, your health meter hits zero, etc.) Switch-case statements are very useful in designing menu-based programs. They define a switch variable and then take different actions based on the value of that variable. If you are working on your own programs you may want to google these other types of control statements and see if they will help you. For our scientific programs, we generally know exactly how many times we want our loops to iterate, making for loops our most common kind of control statement.

1.2 If Statements

The other kind of control statement we use a lot is called an if statement. We use these when we want Octave to run a certain block of code sometimes, but not all the time. The structure of an if statement is:

```
if (condition 1)  
  commands to execute if condition 1 is true;  
elseif (condition 2)  
  commands to execute if condition 2 is true;  
else  
  commands to execute if all conditions are false;  
endif
```

The “if” part tells Octave that an if control statement is coming, and anything between the “if” and the “endif” will be read as part of the control statement. The if control statement can specify any number conditions that will then be evaluated by the computer as either true or false (condition 1, condition 2, etc.; more on this in a minute.) The first condition comes right after the if command. The parentheses are not required, but I usually include them anyway for clarity. There must be at least one condition to evaluate. If the first condition is evaluated as true, Octave will execute the block of associated commands, skip everything else and be finished. If the condition is false, Octave will skip the statements listed under the if part and move on to any elseif blocks it finds. It will execute the commands associated with the first of these elseif conditions that it evaluates as true, then skip everything else and be finished. Note that multiple conditions can be simultaneously true, depending on how you define them. Octave will only execute the statements associated with the first true condition it finds. It is up to us as scientists/programmers to make sure that our if statements proceed logically. This is another case where the computer will happily do something weird and/or physically meaningless if we tell it to. Finally, also have the option of adding an “else” section at the end. The else section does not require a condition. The associated block of commands will execute only if all the if and elseif conditions were false.

The conditions must be defined within the program using what are called comparison operators. You can google up the manual page to see all the options, but you are familiar with these from previous math experience: less than (<), greater than or equal to (>=), etc. Two

comparisons that are particularly useful are “is equal to”, which we specify using two sequential equals signs (==), and “not equal to”, which we specify using an exclamation point followed by an equals sign (!=). Play with the script below to get a sense of how if statements work. Can you get each portion of the if statement to run by manipulating the values assigned to variables x1, x2, and x3? What happens if multiple conditions are true? Can you change the conditions so that the program will execute an “is equal to” (==) comparison and a “not equal to” (!=) comparison?

```
%file created by XXX on YYMMDD  
%tutorial exercise if statements
```

```
x1 = 2;  
x2 = 3;  
x3 = 5;  
  
if (x1 < x2)  
    printf('if executed\n');  
elseif (x2 < x3)  
    printf('elseif executed\n');  
else  
    printf('else executed\n');  
endif
```

2. Formatting

Now that we know about control statements, our programs can get pretty complex. To make them easier to read, we can format our code so that blocks of code that are treated similarly are visually grouped together. So far this hasn’t really been necessary, because the programs have been fairly short. An easy way to do this is by indenting blocks of code that are controlled by common control statements. The computer ignores whitespace in the program, so we can put as many spaces or tabs at the beginning of a line as we want to. You can see this strategy implemented in the code above.

Another good technique is to break your program into chunks that accomplish a common task. You can think of these like paragraphs of code, with each paragraph accomplishing a task the way each paragraph in written work expresses an idea. For example, in a future exercise you will write code that reads data from a file, does so math on the data, and then writes the new

information to a new file. Each of these activities will require a few lines of code. To enhance readability, you might place a few line breaks between each of these three actions.

Commenting is also a helpful way to make your program more readable. There's a bit of debate about whether commenting within your code is "good form" or not. Professional coders tend not to want a lot of comments, as they are so fluent in programming languages that the comments are not really necessary. Scientists tend to be more interested in code explaining what various parts of a program are doing, giving sources for parameters within the program, etc. We aren't quite as fluent in the programming languages, and we are also working with simpler programs.

As you gain experience you will develop your own commenting and formatting style that works for you. At minimum, please use a commented header for each file, and an indenting structure for control statements. You can implement these while you work. Once you get a program running, go back through it and try to read it from the perspective of someone who is encountering your code for the first time. Implement formatting and commenting to make your programs easy to read and understand.

3. Exercises

1. Revisit the Hydrogen energy level table. Create a program that displays your table in the Command Window using a for loop.
2. Revisit your program(s) that verify the various vector and matrix algebra operations, and alter them so that you use loops wherever it is convenient. Make sure that your programs produce the same output when you do the "one-line" version of a calculation and when you use loops. Do not move on until you have these working, writing these programs will help you understand how to get loops and indexing working.
3. You've got a lot of tools at your disposal now. If you have time, try out one of your own ideas for a program. You could write a game of some kind, or write a program that does some math, or messes around with strings. Whatever seems fun to you, just think up a task and try to accomplish it.